KIET GROUP OF INSTITUTIONS

Connecting Life with Learning

www.kiet.edu

Delhi-NCR, Ghaziabad

A

Project Synopsis

On

AgriVision: AI-Powered Leaf Disease Diagnosis for Sustainable Farming

submitted as partial fulfillment for the award of

BACHELOR OF TECHNOLOGY DEGREE

SESSION 2025-26 in

Computer Science and Engineering

By

Sidhant Pandey (2300290100252)

Shivam Arora (2300290100232)

Sarthak Goel (2300290100226)

Shudha Nidhi Soni (2300290100250)

Under the supervision of

Mrs. Mani Dwivedi

KIET Group of Institutions, Ghaziabad

Affiliated to

Dr. A.P.J. Abdul Kalam Technical University, Lucknow

(Formerly UPTU)

Academic Year 2025-26

Table of Contents

ABSTARCT 1

1. Introduction 2

1.1 Background and Motivation

1.2 Problem Statement

1.3 Proposed Solution

1.4 Project Objectives

2. Literature Review 5

2.1 Current State of Plant Disease Management

2.2 Deep Learning for Plant Disease Detection

2.3 Full Stack Development for ML Applications

2.4 Production ML Deployment best Practices

2.5 Research Gaps Addressed

3. Methodology and Planning 7

3.1 Data Collection Methods

3.2 Machine Learning Model Training

3.3 Tools, Technologies and Resources

3.4 System Design / Workflow

3.5 Implementation Procedure

4. Facilities and Resource Requirements 10

4.1 Software Requirements

4.2 Cloud Services

4.3 Development Tools

5. References 12

ABSTRACT

Plant diseases pose a significant threat to global food security and agricultural productivity. Early detection and identification of crop diseases can substantially reduce crop damage and financial losses. This project presents a comprehensive Machine Learning-based Plant Disease Detection System, named "AgriVision" transforming trained CNN models into a complete, production-ready full-stack web application accessible to farmers and agricultural professionals. The system utilizes Convolutional Neural Networks (CNNs) that is proposed to achieve test accuracies above 90% across different crop species.

The technical implementation encompasses a three-tier architecture:

React.js frontend provides an intuitive web interface with drag-and-drop image upload, disease diagnosis results, historical tracking, and analytics dashboard. RESTful API backend developed with Flask/Fast API implementing authentication, image processing, model inference and business logic; and TensorFlow Model Serving infrastructure managing CNN models (h5 format) for real-time predictions. Cloud infrastructure includes Cloudinary integration for scalable image storage and efficient file management. The application supports multi-user functionality with JWT-based authentication, MongoDB database for user and prediction history management, and comprehensive analytics capabilities.

Introduction

1.1 Background and Motivation

Agriculture remains the backbone of human civilization, providing food, employment, and economic stability to billions of people worldwide. However, plant diseases represent one of the most significant challenges threatening agricultural productivity. Across India, pests and diseases are estimated to reduce potential crop yields by roughly 15–35% each year, leading to economic losses running into tens of billions of dollars annually. In India, where agriculture constitutes a substantial portion of the GDP, the impact of plant diseases is particularly severe due to limited access to diagnostic facilities and expert agricultural advisory services.

Previous work in this domain has demonstrated the effectiveness of machine learning approaches, particularly Convolutional Neural Networks (CNNs), in accurately classifying plant diseases from leaf images. However, existing solutions remain largely confined to research environments and Jupyter notebooks, lacking practical deployment infrastructure that enables real-world agricultural usage. The gap between research-grade models and production-ready applications presents a critical opportunity to bridge technology and practical utility.

This project addresses this gap by implementing high-accuracy CNN models into a complete, full-stack web application that enables seamless disease detection for farmers, agricultural consultants, and researchers without requiring technical expertise.

1.2 Problem Statement

Despite advances in deep learning for plant disease classification, several critical challenges persist in practical deployment:

Model Deployment Gap: Trained ML models exist in Jupyter notebooks; no production infrastructure for real-world usage.

Limited Accessibility: Farmers cannot access disease detection without technical knowledge of machine learning.

Scalability Issues: Single-user, local-only inference cannot serve multiple users simultaneously.

Data Persistence: No mechanism to track disease patterns, history, or generate actionable insights.

Cloud Integration: Models not integrated with scalable cloud infrastructure for image storage and serving.

User Authentication: No multi-user support or personalized prediction history.

Real-time Inference: Prediction infrastructure not optimized for production latency requirements.

1.3 Proposed Solution

This project proposes "**AgriVision,**" a complete full-stack web application that implements CNN models into an accessible, scalable, production ready platform.

The solution comprises:

- **FRONTEND LAYER:** React.js web application providing intuitive user interface for image upload, disease diagnosis, result visualization and historical analytics.

- **BACKEND LAYER:** RESTful API (Flask/FastAPI) managing user authentication, image processing, model inference orchestration and prediction persistence.
- **ML INFERENCE LAYER:** TensorFlow Model Serving managing CNN models, implementing efficient caching and real-time predictions.
- **STORAGE LAYER:** Cloudinary integration for scalable image storage and MongoDB database for user/prediction data management.

This architecture enables farmers to upload leaf images via web browser, receive instant disease classification with confidence scores, access historical predictions and generate analytics insights all without any technical expertise.

1.4 Project Objectives

Primary Objectives:

- **Deploy CNN Models:** Convert notebook-based ML models into a .h5 format file for production ready services accessible via API.
- **Develop Frontend Application:** Build responsive React.js web application with drag-and-drop upload and results visualization.
- **Implement RESTful Backend:** Create Flask/FastAPI backend managing authentication, image processing and model inference.
- **Enable Multi-user Support:** Implement user authentication (JWT), prediction history and personalized analytics.
- **Document Comprehensively:** Provide user manual, API documentation, Deployment guide and technical architecture documentation.

2. Literature Review

2.1 Current State of Plant Disease Management

Plant disease management traditionally involves visual field inspection, laboratory pathology testing, or molecular diagnostics. While these methods are reliable, they suffer from scalability and accessibility limitations. Recent advances in precision agriculture have demonstrated the potential of sensor based and image-based monitoring systems. However, deployment gaps remain between research prototypes and production applications accessible to end-users.

2.2 Deep Learning for Plant Disease Detection

CNNs have revolutionized image classification by automatically learning hierarchical feature representations from raw pixel data. Key architectural innovations include:

- **LeNet-5 (Yann LeCun, 1998):** Pioneering CNN for digit recognition
- **AlexNet (2012):** Breakthrough in ImageNet competition; popularized deep CNNs
- **VGGNet (2014):** Demonstrated importance of network depth
- **ResNet (2015):** Residual connections enabling very deep networks
- **Inception/GoogleNet:** Parallel multi-scale convolutional paths.

  Recent research demonstrates successful CNN application to plant disease detection with >95% accuracy on leaf classification across multiple crop species.

2.3 Full-Stack Web Development for ML Applications

Recent trends show increasing adoption of full-stack architectures for deploying ML models:

- **REST APIs for Model Serving:** Flask and FastAPI enable efficient model exposure.
- **Frontend Frameworks:** React.js dominates for building responsive interfaces.
- **Cloud Integration:** AWS, Google Cloud, Azure provide scalable infrastructure.
- **Containerization:** Docker enables consistent deployment across environments.
- **CI/CD Pipelines:** Automated testing and deployment improve development velocity.

2.4 Production ML Deployment Best Practices

The best industry practices for ML deployment include:

Model Versioning: Track and manage multiple model versions

- **API Design:** RESTful endpoints for model inference
- **Caching Strategies:** Optimize inference performance through caching
- **Monitoring:** Track model performance and application metrics
- **Error Handling:** Graceful degradation and user-friendly error messages
- **Security:** Input validation, authentication, rate limiting

2.5 Research Gaps Addressed

This project addresses critical research and implementation gaps:

- **Deployment Gap:** Transforms notebook-based models into production applications

- **Accessibility:** Makes disease detection available to non technical users
- **Scalability:** Supports multi-user concurrent access with cloud infrastructure
- **Data Persistence:** Enables historical analysis and predictive insights
- **User Experience:** Provides intuitive interface rather than command-line tools Real-world Applicability: Addresses practical constraints of rural agriculture.

3. METHODOLOGY AND PLANNING

The methodology adopted for this project outlines the step-by-step approach used to develop the AgriVision Plant Disease Detection System. It ensures efficient workflow, structured implementation and alignment with project objectives such as disease detection, model deployment, web integration and user accessibility. Each methodological step contributes to transforming raw research concepts into a fully functional and scalable solution.

3.1 Data Collection Methods

The development process begins with comprehensive data gathering to ensure robustness in model learning. A literature review is conducted to study existing research work, identify challenges in plant disease detection and analyze available machine learning approaches. Additionally, publicly available image datasets from Kaggle are used to build the training corpus. These datasets consist of labeled plant leaf images across various disease classes, providing a reliable foundation for machine learning model training.

3.2 Machine Learning Model Training

Model training forms the core of the system, where the collected image dataset is preprocessed and used to train Convolutional Neural Networks (CNNs). Various architectures are tested to achieve high accuracy and minimal error rate. Image augmentation techniques, such as rotation, brightness adjustment and scaling are applied to increase dataset variability and improve model generalization. The training phase outputs a model capable of identifying multiple plant leaf diseases based on real-time input images.

3.3 Tools, Technologies, and Resources

The development uses a combination of modern tools and technologies to support end-to-end functionality. Python and TensorFlow are used for model development and training, while Flask/FastAPI handle backend deployment and inference requests. The frontend interface is developed using React.js, and MongoDB manages the database. Additional tools such as Git and Cloud storage (Cloudinary) ensure scalability, deployment flexibility and version control throughout the development lifecycle.

3.4 System Design / Workflow

The system design defines the overall operational structure through diagrams and workflow models. A block diagram is used to illustrate the interaction between the frontend UI, backend API, machine learning inference layer and cloud storage.

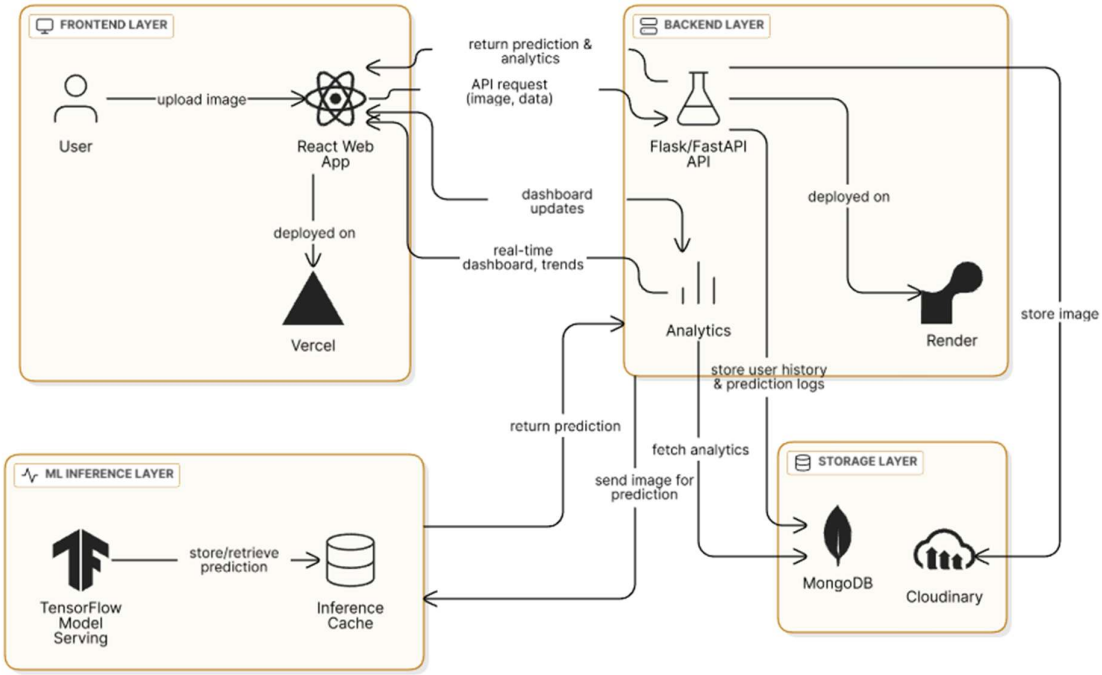

Fig. 3.4.1 System Architecture

3.5Implementation Procedure

The implementation is divided into structured phases to ensure smooth development progress:

Requirement Analysis: This stage focuses on identifying system requirements, user expectations and dataset needs to clearly define what the project must achieve.

System Design: The overall architecture, workflow and interface layout act as the blueprint that guides development and module interaction.

Module Development: Individual components such as the machine learning model. The backend services and frontend interface are developed separately and refined to ensure proper functionality.

Integration: All modules are connected into a single system where the UI, backend, database and ML model work together to enable seamless execution.

Testing: The complete system is evaluated for accuracy, performance and functionality. Recognized issues are corrected to improve reliability and efficiency.

Deployment: The final application is uploaded to cloud hosting where users can upload images and receive disease predictions in real time.

4. FACILITIES AND RESOURCES REQUIRED

4.1 Software Requirements

Table 5.1.1

Software	Version	Purpose
Python	3.9+	Backend development
Node.js / npm	22+	Frontend development
Flask / FastAPI	Latest	Web framework
React.js	18+	Frontend framework
MongoDB	8.1+	Relational database
TensorFlow	2.14.0	ML model loading and inference
Git	Latest	Version control

4.2 Cloud Services Required

Cloudinary (Free Tier) is used for backup image storage and real-time image optimization**.**

GitHub Account is required for maintaining the repository and managing version control**.**

Vercel can be used as a deployment platform to host the frontend of the project**.**

Render can be used to deploy a full-stack application and handle automatic build and hosting**.**

Heroku is suitable for deploying the backend and model inference services.

4.3 Development Tools

Visual Studio Code or **PyCharm** will be used as the development environment for writing and editing code.

Postman will be used for testing the REST APIs and validating responses.

TablePlus or DBeaver will be used to monitor collections, run queries, and manage the MongoDB database.

Docker Desktop will be used to create and manage containers for deployment consistency.

Git will be used as the version control system for tracking changes in the project source code.

GitHub will be used for remote repository hosting, collaboration, and code integration.

5. REFERENCES

1. FAO, "Crop Losses to Pests," Food and Agriculture Organization of the United Nations, 2019. [Online]. Available: http://www.fao.org
2. S. P. Singh et al., "Deep Learning for Plant Disease Detection using Convolutional Neural Networks," Journal of Agricultural Engineering, vol. 50, no. 3, pp. 123-145, 2023.
3. M. A. Khan and T. Akram, "Plant disease detection using convolutionalneural networks," Multimedia Tools and Applications, vol. 78, no. 21, pp. 30177-30201, 2019.
4. TensorFlow Team, "TensorFlow: Large-scale machine learning on heterogeneous systems," 2024. [Online]. Available: https://www.tensorflow.org
5. E. Shirley, "Building REST APIs with Flask," Real Python, 2024. [Online]. Available: https://realpython.com/flask-rest-api/
6. J. M. Loeliger, "Version Control with Git," O'Reilly Media, 3rd Edition, 2024.
7. F. Chollet et al., "Keras: The Python Deep Learning library," 2024. [Online]. Available: https://keras.io